

◆ PROJECT HANDOVER DOCUMENT ◆

CYBER NOTES

Laravel Edition — Learning & Practice Guide

FRAMEWORK

Laravel 12

LANGUAGE

PHP 8.3

DATABASE

SQLite

AUTH

Breeze

STYLING

Tailwind + DaisyUI

LOCAL URL

php-project.test

GitHub: github.com/unique-mechanic/php-scribble | Generated: July 2026

◆ CONTENTS

01	Project Overview	pg 2
02	Tech Stack	pg 2
03	Setup Guide	pg 3
04	Project Structure	pg 4
05	Database Schema	pg 5
06	Routes Reference	pg 5
07	How It Works — MVC Flow	pg 6
08	Key Laravel Concepts	pg 7
09	Code Walkthrough	pg 8
10	Artisan Commands	pg 10
11	Practice Exercises	pg 11

01 — OVERVIEW

Project Overview

CyberNotes is a full-stack note-taking web application with a cyberpunk aesthetic. It was originally built as a *custom PHP MVC framework from scratch* and was later migrated to **Laravel** — making it an ideal learning project because you can see both approaches side by side in the git history.

The app allows users to register, log in, and manage their own private notes. Every user only sees their own data. It demonstrates the core building blocks of any real Laravel application.



What the app does

Users register/login and get a private notes database. They can create, read, update and



What you will learn

Laravel routing, Eloquent ORM, Blade templates, authentication with Breeze, form

delete (CRUD) their notes. Each note has a body, timestamp, and belongs to a user.

validation, middleware, flash messages, and resource controllers.



Architecture

MVC — Models handle data, Controllers handle logic, Views handle display. Routes connect URLs to controllers. Middleware guards protected pages.



Security features

CSRF protection on all forms, authenticated routes via middleware, user ownership checks (403 if wrong user), hashed passwords via bcrypt.

Tech Stack

LAYER	TECHNOLOGY	PURPOSE	VERSION
Backend Framework	Laravel	Routing, ORM, Auth, Validation	12.x
Language	PHP	Server-side logic	8.3+
Database	SQLite	File-based SQL database	3.x
Auth Scaffolding	Laravel Breeze	Login, Register, Logout pages	2.x
CSS Framework	Tailwind CSS	Utility-first styling (via CDN)	3.x
UI Components	DaisyUI	Pre-built Tailwind components	4.x
Fonts	Orbitron + Space Mono	Cyberpunk typography	Google Fonts
Local Dev	Laravel Herd	Local server + PHP manager	–
Template Engine	Blade	PHP-powered HTML templates	built-in

Setup Guide



Prerequisites

You need PHP 8.3+, Composer, and Node.js installed. Laravel Herd is recommended for macOS. Git is required to clone the repo.

Step-by-step Installation

1

Clone the repository

Get the code onto your machine from GitHub

```
git clone https://github.com/unique-mechanic/php-scribble.git
cd php-scribble
```

2

Install PHP dependencies

Composer downloads all Laravel packages into the vendor/ folder

```
composer install
```

3

Install JavaScript dependencies & build assets

npm downloads Tailwind/Vite tools and compiles CSS/JS

```
npm install
npm run build
```

4

Create your environment file

Copy the example .env file and generate a unique app key

```
cp .env.example .env
```

```
php artisan key:generate
```

5 Create the SQLite database & run migrations

This creates the database file and builds all tables

```
touch database/database.sqlite  
php artisan migrate
```

6 Run the development server

Choose one: built-in PHP server or Laravel Herd

```
# Option A: PHP built-in server  
php -S localhost:8000 -t public  
  
# Option B: Laravel Herd (macOS)  
# Just place the folder in ~/Herd/ or run: herd link
```



You're up and running!

Visit <http://localhost:8000> (or <http://php-scribble.test> with Herd). Register a new account and start creating notes.

Understanding the .env File

The `.env` file holds environment-specific configuration. It is **never committed to git** (it's in `.gitignore`) because it can contain secrets.

```
# App identity  
APP_NAME=CyberNotes  
APP_URL=http://localhost:8000  
  
# Database - SQLite (file-based, no server needed)  
DB_CONNECTION=sqlite  
# DB_DATABASE defaults to database/database.sqlite  
  
# Change to mysql for MySQL/MariaDB:  
# DB_CONNECTION=mysql  
# DB_HOST=127.0.0.1
```

```
# DB_DATABASE=cybernotes  
# DB_USERNAME=root  
# DB_PASSWORD=
```

Project Structure

Laravel follows a well-defined directory structure. Here are the folders you will work in most often:

```

php-scribble/ | ├── app/ ← Your application code lives here | ├── Http/ | | |
Controllers/ | | | |── NoteController.php ← CRUD for notes → start here | | | |──
Auth/ ← Breeze auth controllers | | | |── ProfileController.php | | | |── Requests/ ←
Form request validation classes | | | |── Models/ | | | |── Note.php ← Note model (Eloquent)
→ | | | |── User.php ← User model (Eloquent) | | | |── database/ | | | |── migrations/ ←
Database table blueprints | | | |── ..._create_users_table.php | | | |──
..._create_notes_table.php ← Notes schema → | | | |── database.sqlite ← The actual
database file (gitignored) | | | |── seeders/ ← Seed fake data for testing | | | |──
resources/ | | | |── views/ ← Blade HTML templates | | | |── layouts/ | | | |──
cyber.blade.php ← Main cyberpunk layout → | | | |── guest.blade.php ← Auth pages
layout | | | |── notes/ ← Notes views → | | | |── index.blade.php ← List all notes | | | |──
create.blade.php ← Create form | | | |── show.blade.php ← View single note | | | |──
edit.blade.php ← Edit form | | | |── auth/ ← Login, register pages | | | |──
welcome.blade.php ← Homepage | | | |── routes/ | | | |── web.php ← All web routes → start
here | | | |── auth.php ← Breeze auth routes | | | |── .env ← Environment config (never
commit!) | | | |── .env.example ← Template for .env | | | |── composer.json ← PHP dependencies
| | | |── artisan ← Laravel CLI tool

```



◆ Start with these files

If you are new to Laravel, read these files in order: routes/web.php → app/Http/Controllers/NoteController.php → app/Models/Note.php → resources/views/notes/index.blade.php

Database Schema

users table

COLUMN	TYPE	NOTES
id	bigint, PK, auto-increment	Primary key
name	varchar(255)	Display name
email	varchar(255), unique	Login identifier
email_verified_at	timestamp, nullable	Email verification
password	varchar(255)	Bcrypt hash — never plain text
remember_token	varchar(100), nullable	"Remember me" cookie
created_at	timestamp	Auto-managed by Laravel
updated_at	timestamp	Auto-managed by Laravel

notes table

COLUMN	TYPE	NOTES
id	bigint, PK, auto-increment	Primary key
user_id	bigint unsigned, FK	References users.id — CASCADE delete
body	text	The note content
created_at	timestamp	Auto-managed by Laravel
updated_at	timestamp	Auto-managed by Laravel



Relationship:

One User has many Notes. When a user is deleted, all their notes are automatically deleted too (CASCADE). This is defined in the migration via `→constrained()→onDelete('cascade')`.

Routes Reference

METHOD	URL	CONTROLLER METHOD	DESCRIPTION	AUTH?
GET	/	Closure (welcome view)	Homepage	Public
GET	/login	AuthenticatedSession@create	Login form	Public
POST	/login	AuthenticatedSession@store	Process login	Public
POST	/logout	AuthenticatedSession@destroy	Logout	Auth
GET	/register	RegisteredUser@create	Register form	Public
POST	/register	RegisteredUser@store	Create account	Public
GET	/notes	NoteController@index	List all user's notes	Auth
GET	/notes/create	NoteController@create	Show create form	Auth
POST	/notes	NoteController@store	Save new note	Auth
GET	/notes/{note}	NoteController@show	View a note	Auth
GET	/notes/{note}/edit	NoteController@edit	Show edit form	Auth
PATCH	/notes/{note}	NoteController@update	Save edits	Auth
DELETE	/notes/{note}	NoteController@destroy	Delete a note	Auth



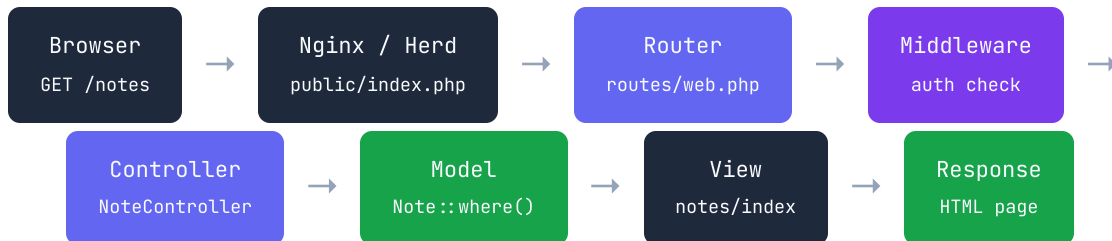
Resource Controllers

All 7 note routes are registered with a single line: `Route::resource('notes', NoteController::class);` Laravel generates all the CRUD routes automatically following REST conventions.

How It Works — MVC Flow

Request Lifecycle

Every page visit in Laravel follows the same journey from browser to response:



Creating a Note — Full Flow

Step 1 — User visits `GET /notes/create`

```
// routes/web.php — router matches the URL
Route::resource('notes', NoteController::class);
```

Step 2 — Controller returns the form view

```
// NoteController.php
public function create()
{
    return view('notes.create'); // renders resources/views/notes/create.blade.php
}
```

Step 3 — User fills form and submits `POST /notes`

```
{{-- resources/views/notes/create.blade.php --}}
<form method="POST" action="{{ route('notes.store') }}">
    @csrf {{-- Security token --}}
    <textarea name="body"></textarea>
    <button type="submit">Save</button>
</form>
```

Step 4 — Controller validates, saves, and redirects

```
// NoteController.php
public function store(Request $request)
{
    $request->validate(['body' => 'required|string|min:1']);

    Note::create([
        'body' => $request->body,
        'user_id' => Auth::id(), // attach to logged-in user
    ]);

    return redirect()->route('notes.index')
        ->with('success', 'Note created!'); // flash message
}
```

Step 5 — Model saves to database via Eloquent ORM

```
// app/Models/Note.php - tells Laravel which fields are saveable
protected $fillable = ['body', 'user_id']; // mass-assignment protection
```

Key Laravel Concepts

1. Eloquent ORM — Talking to the Database

Eloquent lets you interact with the database using PHP objects instead of raw SQL queries.

```
// Raw SQL (old way)
SELECT * FROM notes WHERE user_id = 5;

// Eloquent (Laravel way) — same result, much more readable
$notes = Note::where('user_id', 5)→get();

// Even better — use the relationship
$notes = auth()→user()→notes→get();

// Create, update, delete
Note::create(['body' ⇒ 'Hello', 'user_id' ⇒ 1]);
$note→update(['body' ⇒ 'Updated']);
$note→delete();
```

2. Blade Templates — Dynamic HTML

Blade is Laravel's template engine. It adds special syntax to HTML files for loops, conditionals, and layouts.

```
{{-- Output a variable (auto-escaped, safe from XSS) --}}
{{ $note→body }}
```

```
{{-- Conditionals --}}
@if(session('success'))
    <div>{{ session('success') }}</div>
@endif
```

```
{{-- Loops --}}
@foreach($notes as $note)
    <p>{{ $note→body }}</p>
@endforeach
```

```
{{-- Inherit a layout --}}
```

```
@extends('layouts.cyber')
@section('content')
    <h1>My Page</h1>
@endsection
```

3. Middleware — Route Guards

Middleware runs before your controller. The `auth` middleware redirects unauthenticated users to the login page.

```
// routes/web.php - protect all note routes
Route::middleware(['auth'])->group(function () {
    Route::resource('notes', NoteController::class);
});
```

4. Validation

```
// In a controller - fails and returns errors automatically
$request->validate([
    'body' => 'required|string|min:1|max:5000',
    'email' => 'required|email|unique:users',
]);

{{-- In Blade - show error messages --}}
@error('body')
    <p class="error">{{ $message }}</p>
@enderror
```

5. Flash Messages

```
// Set in controller after an action
return redirect()->route('notes.index')->with('success', 'Note deleted!');

// Read in Blade (only shows once, then gone)
@if(session('success'))
    <div>{{ session('success') }}</div>
@endif
```

6. Authorization — Ownership Check

```
// Prevent user A from editing/deleting user B's notes
abort_if($note→user_id ≠ Auth::id(), 403);
// Throws a 403 Forbidden error if the note doesn't belong to the logged-in user
```

Code Walkthrough

NoteController.php — The Heart of the App

```
class NoteController extends Controller
{
    // GET /notes - list all notes for the logged-in user
    public function index()
    {
        $notes = Note::where('user_id', Auth::id())->latest()->get();
        return view('notes.index', compact('notes'));
        // compact('notes') = ['notes' => $notes]
    }

    // GET /notes/create - show the blank create form
    public function create()
    {
        return view('notes.create');
    }

    // POST /notes - save the new note
    public function store(Request $request)
    {
        $request->validate(['body' => 'required|string|min:1']);
        Note::create(['body' => $request->body, 'user_id' => Auth::id()]);
        return redirect()->route('notes.index')->with('success', 'Note created!');
    }

    // GET /notes/{note} - show one note (route model binding)
    public function show(Note $note) // ← Laravel auto-fetches the Note by ID!
    {
        abort_if($note->user_id !== Auth::id(), 403); // ownership check
        return view('notes.show', compact('note'));
    }

    // DELETE /notes/{note} - delete and redirect
    public function destroy(Note $note)
    {
        abort_if($note->user_id !== Auth::id(), 403);
        $note->delete();
    }
}
```

```
        return redirect()→route('notes.index')→with('success', 'Note deleted!');
    }
}
```

Note.php Model

```
class Note extends Model
{
    protected $fillable = ['body', 'user_id']; // allow mass assignment for these fields

    // Define relationship: Note belongs to a User
    public function user()
    {
        return $this→belongsTo(User::class);
    }
}

// Usage examples:
$note→user;           // get the User who owns this note
$user→notes;          // get all Notes for a user (hasMany defined on User)
```

routes/web.php

```
use App\Http\Controllers\NoteController;

Route::get('/', fn() ⇒ view('welcome'))→name('home');

// All 7 CRUD routes for notes, guarded by auth middleware
Route::middleware(['auth'])→group(function () {
    Route::resource('notes', NoteController::class);
    // Generates: index, create, store, show, edit, update, destroy
});

require __DIR__.'./auth.php'; // Breeze auth routes
```


Artisan Commands Cheatsheet

Artisan is Laravel's command-line tool. Run all commands from the project root.

Development

COMMAND	WHAT IT DOES
<code>php artisan serve</code>	Start built-in dev server at localhost:8000
<code>php artisan route:list</code>	List all registered routes
<code>php artisan tinker</code>	Interactive PHP REPL — test Eloquent queries live
<code>php artisan about</code>	Show app info (PHP version, DB, etc.)

Database

COMMAND	WHAT IT DOES
<code>php artisan migrate</code>	Run pending migrations (create/alter tables)
<code>php artisan migrate:fresh</code>	⚠ Drop ALL tables and re-run migrations
<code>php artisan migrate:fresh --seed</code>	Re-create tables and fill with fake data
<code>php artisan migrate:status</code>	Show which migrations have been run
<code>php artisan db:seed</code>	Run database seeders

Code Generation (make:)

COMMAND	WHAT IT CREATES
<code>php artisan make:model Post -m</code>	Model + migration file
<code>php artisan make:controller PostController --resource</code>	Resource controller with all 7 methods
<code>php artisan make:migration add_title_to_notes</code>	New migration file

COMMAND	WHAT IT CREATES
<code>php artisan make:seeder NotesSeeder</code>	Seeder to fill fake data
<code>php artisan make:middleware CheckAge</code>	Custom middleware class
<code>php artisan make:request StoreNoteRequest</code>	Form request validation class

Cache

COMMAND	WHAT IT DOES
<code>php artisan config:clear</code>	Clear cached config (run after editing .env)
<code>php artisan cache:clear</code>	Clear application cache
<code>php artisan view:clear</code>	Clear compiled Blade views
<code>php artisan optimize:clear</code>	Clear all caches at once



migrate:fresh destroys all data!

Never run `migrate:fresh` in production. It drops and recreates every table, deleting all records.

Practice Exercises

The best way to learn Laravel is to extend this project. Here are exercises ordered by difficulty:

EASY 1. Add a Title Field to Notes

Currently notes only have a body. Add a title column to the notes table.

- Create a migration: `php artisan make:migration add_title_to_notes_table`
- Add title to the `$fillable` array in the Note model
- Update the create and edit forms to include a title input
- Update the index and show views to display the title

EASY 2. Add Pagination to the Notes List

When a user has many notes, the page gets long. Add pagination to show 10 notes per page.

- In `NoteController@index` change `→get()` to `→paginate(10)`
- In the view, add `{{ $notes→links() }}` below the notes grid

MEDIUM 3. Add a Search Bar

Let users search their notes by keyword.

- Add a search form (GET method) to the notes index view
- In the controller, check for a search query parameter and add a `→where('body', 'LIKE', "%{$search}%")` clause
- Keep the search term in the input field after submission

MEDIUM 4. Add Note Categories / Tags

Let users tag notes as Work, Personal, Ideas, etc.

- Add a category column (string, nullable) to the notes migration
- Add a dropdown select to the create/edit forms
- Filter notes by category in the sidebar (the filter UI already exists!)

MEDIUM 5. Add a User Profile Page

Breeze already generates a profile page. Customise it with the cyberpunk theme.

- Visit `/profile` — Breeze already built this!
- Update `resources/views/profile/edit.blade.php` to use the cyber layout
- Add a "notes count" stat to the profile

HARD 6. Build a REST API for Notes

Create a JSON API so the notes can be accessed programmatically.

- Create an `Api/NoteController.php` that returns JSON responses
- Define routes in `routes/api.php`
- Use Laravel Sanctum (already installed) for API token authentication
- Test with `curl` or Postman/Bruno

HARD 7. Add Database Seeding with Faker

Populate the database with realistic fake data for testing.

- Create a `NoteFactory`: `php artisan make:factory NoteFactory`
- Use `fake()→paragraph()` for the body field
- Create a `NotesSeeder` and call it from `DatabaseSeeder`
- Run: `php artisan migrate:fresh --seed`



Go further

After completing these exercises, explore: Laravel Queues (for background jobs), Laravel Mail (send emails on registration), Laravel Events & Listeners, or replace SQLite with MySQL for production.